

Voice-Based AI Assistant Using Local API Integration

Shashwat Malviya

Dept. of Computer Science and Engg.
Graphic Era (Deemed to be University)
Dehradun, India
shashwat.malviya123@gmail.com

Ritendra Negi

Dept. of Computer Science and Engg.
Graphic Era (Deemed to be University)
Dehradun, India
retendranegi@gmail.com

Siddhant Srivastava

Dept. of Computer Science and Engg.
Graphic Era (Deemed to be University)
Dehradun, India
sidhantsrivastav13@gmail.com

Yash Rawat

Dept. of Computer Science and Engg.
Graphic Era (Deemed to be University)
Dehradun, India
yashrawats005@gmail.com

Abstract—Voice-based artificial intelligence assistants have become an increasingly common interface between humans and computing systems. Commercially available assistants such as Amazon Alexa, Google Assistant, and Apple Siri have demonstrated the practical appeal of natural language voice interaction; however, these systems are fundamentally reliant on cloud infrastructure for both processing and storage. This reliance raises well-documented concerns regarding user privacy, the potential for transcript misuse, high-latency responses, and susceptibility to network disruptions. Additionally, large language models (LLMs) deployed in cloud-based assistants have been observed to produce factually incorrect or fabricated outputs, a phenomenon commonly described as hallucination.

This paper presents the design and development of a voice-based AI assistant that addresses these limitations through a locally hosted, modular architecture. The proposed system integrates automatic speech recognition (ASR) using the Whisper model, transformer-based language generation, and Retrieval-Augmented Generation (RAG) to produce contextually accurate, knowledge-grounded responses without relying on external cloud services. Audio data captured during interaction is encrypted using AES-256 before storage, and communication between system modules is managed via a FastAPI-based local API, ensuring that no user data leaves the host machine.

The system further incorporates an intent classification layer to distinguish between conversational queries and direct OS automation commands, enabling the assistant to perform desktop tasks alongside natural language interaction. Expected outcomes include improved response accuracy through RAG-based grounding, enhanced data privacy through local processing and encryption, and a modular design that facilitates future extensibility. The system holds practical value for users in research, healthcare, legal, and enterprise settings where data confidentiality is a primary concern [1]–[3].

Index Terms—Voice Assistant, Speech Recognition, Retrieval-Augmented Generation, Local API, AES Encryption, FastAPI, Large Language Models, Intent Classification.

I. INTRODUCTION

The rapid advancement of artificial intelligence over the past decade has significantly altered how humans interact with

computing devices. Among the most visible manifestations of this shift is the emergence of voice-based AI assistants, which allow users to interact with systems using natural spoken language rather than conventional input methods such as keyboards or touchscreens. Early deployments of such assistants, including Apple’s Siri introduced in 2011 and Amazon Echo launched in 2014, established the foundational expectations for voice-activated computing: always-available, low-latency, natural language interaction with underlying digital services [4], [5].

Despite the widespread adoption of these systems, a growing body of research has documented significant shortcomings in commercially deployed voice assistants. The most structurally significant of these is cloud dependency. Current commercial assistants transmit raw audio or partially processed speech data to remote servers for recognition and inference [6]. While this approach allows providers to leverage substantial computational resources, it introduces several problems. First, users have no direct control over their audio data once it is transmitted. Documented incidents involving the inadvertent storage and human review of private voice recordings have raised serious concerns about the boundaries of informed user consent [7]. Second, network-dependent architectures introduce non-trivial latency and become entirely non-functional in offline or low-bandwidth environments.

A second class of limitations relates to the language models that underpin these assistants. Modern LLMs, while capable of highly fluent and contextually appropriate text generation, are prone to producing responses that are factually incorrect or internally inconsistent with the user’s actual knowledge environment [8]. This hallucination problem is particularly problematic in domains where factual accuracy carries practical consequences—such as medical information retrieval, legal document summarisation, or technical troubleshooting. Standard LLM inference does not query the user’s personal

documents or locally stored knowledge, meaning responses are generated purely from parameters learned during training, which may be outdated or contextually misaligned [9].

A third limitation concerns system modularity. Commercially deployed voice assistants are typically closed ecosystems that offer limited ability for end-users or developers to modify the processing pipeline, substitute individual components, or integrate custom knowledge sources. This constrains adaptability and inhibits deployment in specialised domains.

The proposed system addresses these limitations through a locally hosted, modular voice assistant architecture. The system processes speech entirely on the host machine, using the OpenAI Whisper model for accurate offline speech recognition [10]. An intent classification layer routes user input either to an OS automation module or to a language reasoning pipeline. For queries requiring factual grounding, a Retrieval-Augmented Generation (RAG) module retrieves relevant passages from a local vector store before passing enriched context to a locally running LLM [11]. All audio and transcripts are encrypted at rest using AES-256, and inter-module communication is handled via FastAPI endpoints running on the local network interface.

The motivation for this work is grounded in the observation that local processing is no longer infeasible for typical computing hardware. The availability of quantised LLMs capable of running on consumer GPUs, efficient vector databases such as FAISS and ChromaDB, and mature Python frameworks for audio processing and API design means that a fully local, privacy-preserving voice assistant is now an achievable engineering objective [12], [13]. The system described in this paper demonstrates one concrete realisation of this objective, with an architecture designed to be extended and adapted for a variety of application domains.

The remainder of this paper is structured as follows: Section II surveys the existing literature relevant to voice assistants, language models, RAG, and secure data processing. Section III describes the system methodology and implementation details across all modules. Section IV presents anticipated results and a discussion of expected outcomes. Section V outlines planned future directions for the work.

II. LITERATURE REVIEW

A. Voice Assistants and Speech Recognition

The development of voice-based human–computer interaction systems has followed a trajectory shaped by advances in acoustic modelling, language modelling, and hardware capability. Early automatic speech recognition (ASR) systems relied on hidden Markov models (HMMs) combined with Gaussian mixture models (GMMs) for phoneme-level acoustic modelling, an approach that remained dominant for several decades following its formalisation in the 1980s [14]. The transition to deep neural networks for acoustic modelling in the early 2010s produced substantial accuracy improvements, enabling the commercial deployment of voice assistants at scale [4].

Radford et al. introduced the Whisper model in 2022, demonstrating that a transformer-based ASR system trained on 680,000 hours of weakly supervised multilingual audio could achieve near-human-level transcription accuracy across diverse acoustic conditions and languages without fine-tuning [10]. Whisper’s open-weight release made it the most practically significant development in offline ASR in recent years, enabling accurate speech recognition to be deployed entirely on local hardware. Prior to Whisper, offline ASR options such as CMU Sphinx and Kaldi offered acceptable accuracy for limited vocabulary domains but struggled with the speaker and acoustic diversity typical of real-world usage [15].

Surveys of commercial voice assistant architectures document a consistent pattern: audio captured on device is streamed to cloud endpoints for recognition, intent parsing, and response generation, with only a thin client layer maintained locally [6]. This design prioritises scalability but systematically excludes offline use cases and creates a privacy surface that has been exploited in several documented incidents [7].

B. Transformer-Based Language Models

The transformer architecture, proposed by Vaswani et al. in 2017, introduced the self-attention mechanism as a replacement for recurrence in sequence modelling [1]. This architectural shift enabled parallelisation during training and facilitated scaling to model sizes that had previously been computationally intractable. The subsequent introduction of BERT by Devlin et al. demonstrated that bidirectional pre-training on large text corpora produced language representations that could be fine-tuned to achieve state-of-the-art performance across a wide range of NLP benchmarks [16].

The GPT family of models, developed by OpenAI, extended the transformer paradigm to autoregressive language generation, producing models capable of coherent long-form text generation from a brief prompt [17]. Subsequent scaling experiments documented predictable relationships between model size, training compute, and downstream task performance, motivating the development of increasingly large models [18]. However, the practical deployment of multi-billion-parameter models on commodity hardware was constrained by memory requirements until the introduction of quantisation techniques such as GGUF and GPTQ, which reduced model footprint sufficiently to permit inference on consumer-grade GPUs [19]. Locally deployable models such as LLaMA 2 and Mistral have since made on-device LLM inference a practical proposition for developers [20].

Despite their generative capabilities, all autoregressive LLMs share a fundamental limitation: their responses are generated from parameters fixed at training time and therefore cannot reflect information introduced after the training cutoff or present in the user’s local document environment. This limitation is the primary motivation for the RAG paradigm described in Section II-C.

C. Retrieval-Augmented Generation

Lewis et al. introduced Retrieval-Augmented Generation (RAG) in 2020 as a method for combining parametric language model knowledge with non-parametric retrieval from a document index [2]. In the RAG framework, a user query is encoded into a dense vector representation and used to retrieve the most semantically relevant passages from an indexed document corpus. These retrieved passages are concatenated with the original query and passed as context to a generative model, enabling the model to produce responses grounded in retrieved evidence rather than purely in learned parameters.

Gao et al. provided a systematic survey of RAG techniques, categorising approaches as naive, advanced, or modular RAG based on the sophistication of the retrieval and context integration pipeline [21]. The survey identified several recurring challenges, including retrieval precision, the handling of conflicting retrieved passages, and the optimal integration of retrieved content into the generation prompt. Dense retrieval approaches, particularly those employing bi-encoder models based on sentence transformers, have been shown to outperform sparse approaches such as BM25 for semantically complex queries [22].

The application of RAG to local assistant systems has been explored in the context of enterprise knowledge management, where the ability to query internal document repositories without transmitting sensitive data to external services is of practical importance [11]. Vector database implementations including FAISS, developed by Johnson et al. at Meta AI, and ChromaDB provide efficient approximate nearest-neighbour search over high-dimensional embedding spaces, enabling millisecond-scale retrieval from document corpora of moderate size [23].

D. Voice Authentication and Biometric Systems

Voice authentication uses acoustic features of the human voice—including fundamental frequency, formant structure, and spectral envelope—as a biometric identifier. Speaker verification systems based on deep neural networks have achieved equal error rates below 1% on standard benchmark datasets, making voice a viable modality for lightweight authentication in controlled acoustic conditions [24]. However, voice biometrics are susceptible to replay attacks and voice cloning, and their security properties degrade significantly in noisy environments or with low-quality microphone hardware [25]. The proposed system incorporates optional face-based authentication as a less acoustically sensitive alternative, implemented using local face detection and embedding comparison without external API calls.

E. Secure Data Processing and Encryption

The encryption of sensitive data at rest and in transit is a well-established requirement in secure system design. The Advanced Encryption Standard (AES), standardised by NIST in 2001, remains the dominant symmetric cipher for data at rest due to its combination of security and computational efficiency [26]. The use of AES-256 in Galois/Counter Mode

(GCM) provides both confidentiality and integrity guarantees and is appropriate for the encryption of audio files and transcripts in the context of this system. Key management is a recognised challenge in local encryption schemes; the proposed system uses a locally stored key derived from user credentials, avoiding the complexities of public key infrastructure while maintaining adequate security for the threat model of a personal assistant application.

Previous work on secure voice processing systems has noted the importance of minimising the duration for which decrypted audio data remains in memory during processing, a principle sometimes described as in-memory transience [27]. The proposed system decrypts audio to a temporary in-memory buffer for the duration of ASR processing and does not write decrypted audio to disk.

F. Research Gap

The reviewed literature identifies no existing open-source system that integrates locally hosted ASR, RAG-enhanced LLM reasoning, OS automation, and AES-encrypted storage within a unified FastAPI-based modular architecture for a consumer or research computing environment. Commercial assistants address the usability requirements but sacrifice privacy and offline capability. Academic work on RAG and LLMs focuses predominantly on server-scale deployments. Local assistant frameworks such as Home Assistant and Open Assistant provide partial functionality but lack integrated RAG, encryption, or a structured local API architecture. The system proposed in this paper addresses this gap by providing an end-to-end locally hosted voice assistant with privacy-by-design properties.

III. METHODOLOGY

This section describes the design and implementation of the proposed voice-based AI assistant. The system is implemented in Python 3.11 and comprises a set of loosely coupled modules communicating via a FastAPI-based local REST API. The following subsections describe each module in detail.

A. System Architecture

The system is organised as a directed processing pipeline in which each stage performs a well-defined transformation on the user's input before passing it to the next stage. The high-level flow is as follows: the user initiates an interaction either through a graphical interface or a hotword trigger. Optionally, a face authentication check is performed using a locally deployed face recognition model before processing proceeds. The system then captures microphone audio, applies preprocessing to normalise levels and reduce background noise, and passes the processed audio buffer to the encryption layer for storage. A copy of the audio is simultaneously forwarded to the speech-to-text module.

The transcript produced by the ASR module is passed to the intent classification layer, which determines whether the input constitutes an OS automation command, a retrieval-augmented query, or a direct generative query. Based on this classification,

the input is routed to the appropriate downstream module: the automation module for system commands, or the RAG pipeline followed by the LLM for knowledge queries. The generated response is converted to speech by the text-to-speech (TTS) module and played back through the system's audio output. All inter-module communication occurs via FastAPI endpoints bound to the localhost interface, ensuring that no traffic escapes the host machine's network stack.

This modular architecture offers several practical advantages. Each component can be independently upgraded, replaced, or disabled without requiring modifications to adjacent modules. The use of a local REST API as the integration layer means that modules can in principle be implemented in different languages or run as separate processes, and the system can be extended to support multi-machine local network deployments with minimal architectural changes.

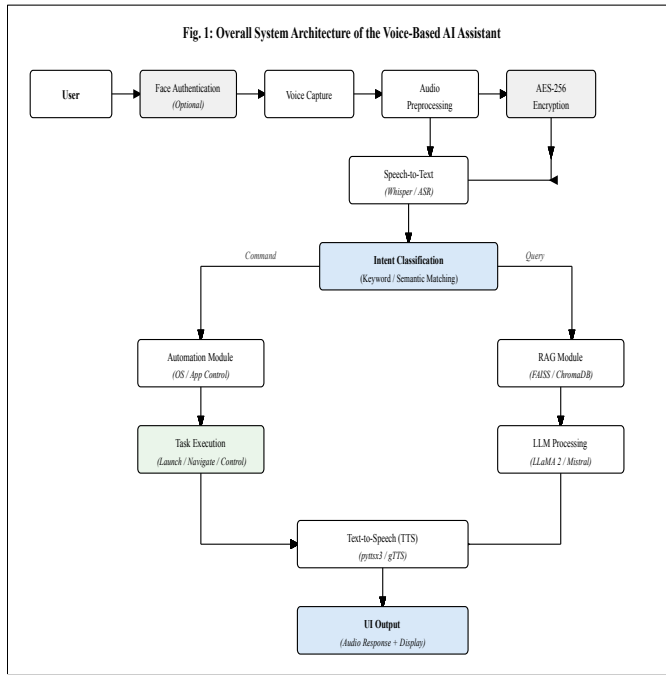


Fig. 1. Overall system architecture of the proposed privacy-preserving, locally hosted voice assistant, illustrating the complete processing pipeline from user input to audio output.

B. Speech Processing Module

Audio capture is performed using the PyAudio library, which provides a cross-platform interface to the host system's audio input devices. The system records audio in configurable chunks of 1024 samples at a 16,000 Hz sampling rate, the input format expected by the Whisper model. A voice activity detection (VAD) step, implemented using the WebRTC VAD algorithm, monitors the incoming audio stream and triggers recording when speech is detected, terminating the recording after a configurable period of silence.

Before transcription, the captured audio is pre-processed to improve recognition accuracy. A high-pass filter with a cutoff frequency of 80 Hz attenuates low-frequency noise from

sources such as HVAC systems and electrical hum. Amplitude normalisation scales the audio signal to a consistent peak level, compensating for variation in microphone sensitivity and speaker distance. For longer recordings, an optional mel spectrogram visualisation is generated using the librosa library, which can be used for diagnostic purposes.

Transcription is performed using the OpenAI Whisper model in its base or small configuration depending on available hardware. Whisper is a transformer-based ASR model that processes 30-second audio windows and produces time-stamped word-level transcripts [10]. The model is loaded locally using the openai-whisper Python package, which downloads and caches model weights on first use. The system also supports the SpeechRecognition library as a fallback ASR backend for low-resource environments, using the Google Speech API when network access is available or CMU PocketSphinx for fully offline operation [15].

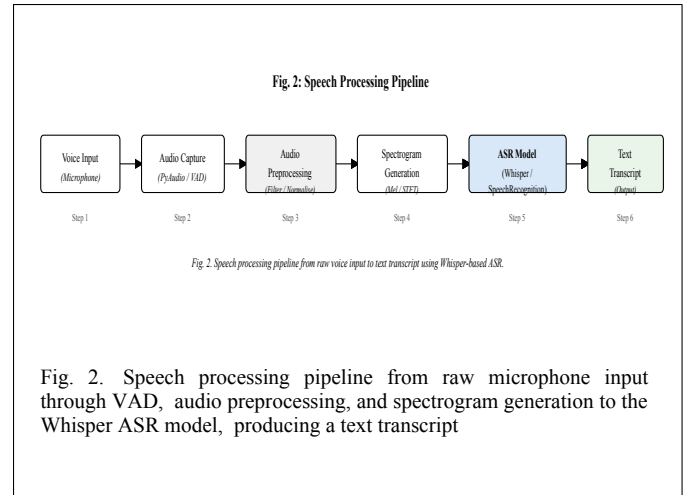


Fig. 2. Speech processing pipeline from raw voice input to text transcript using Whisper-based ASR.

Fig. 2. Speech processing pipeline from raw microphone input through VAD, audio preprocessing, and spectrogram generation to the Whisper ASR model, producing a text transcript

C. Intent Classification

The intent classification module determines the appropriate processing path for each transcribed utterance. In the current implementation, classification is performed using a rule-based keyword matching approach. A curated dictionary of command keywords and phrases—including terms such as “open”, “close”, “search”, “navigate”, “volume”, “shutdown”, and “launch”—is compared against the normalised transcript. If a keyword match is detected above a configurable confidence threshold, the utterance is classified as an automation command and routed to the automation module. Utterances

that do not trigger a keyword match are classified as queries and routed to the RAG-LLM pipeline.

Keyword-based classification is computationally inexpensive and interpretable, making it appropriate for a first implementation where reliability and predictability are valued over nuance. However, it is recognised that this approach cannot correctly handle paraphrased commands, negations, or utterances that combine command and query elements. A semantic intent classification approach using a fine-tuned BERT-based classifier or a zero-shot classification model such as BART-MNLI is identified as a priority for future development.

D. Retrieval-Augmented Generation (RAG)

The RAG module is responsible for grounding the language model's responses in content drawn from the user's local document corpus. The module operates in two phases: an offline indexing phase and an online retrieval phase.

During indexing, documents in supported formats (PDF, DOCX, TXT, and Markdown) are loaded from a designated local knowledge directory. Each document is segmented into overlapping chunks of approximately 500 tokens with a 50-token overlap between consecutive chunks, using the text-splitting utilities provided by the LangChain framework [3]. This chunking strategy balances the trade-off between providing sufficient context within each chunk and maintaining retrieval granularity. Each chunk is then encoded into a dense vector representation using a sentence transformer model, specifically the `all-MiniLM-L6-v2` model from the sentence-transformers library [22], which produces 384-dimensional embeddings. The resulting embedding vectors are stored in a FAISS index alongside the corresponding text chunks and document metadata [23].

During online retrieval, the user's transcribed query is encoded using the same sentence transformer model. A nearest-neighbour search is performed against the FAISS index to identify the top- k most semantically similar chunks, where k is a configurable parameter defaulting to five. The retrieved chunks are ranked by cosine similarity score and the top results are concatenated into a context string. This context string is prepended to the user's query in the prompt template passed to the language model, following the standard RAG prompting pattern: "Given the following context: [retrieved passages]. Answer the following question: [user query]."

The RAG module substantially reduces the incidence of hallucination for queries that fall within the scope of the indexed document corpus. By providing the language model with retrieved textual evidence directly relevant to the query, the model is guided to produce responses that are grounded in the supplied context rather than in potentially outdated or inaccurate parametric knowledge. Queries for which no sufficiently similar documents are found in the index are handled by the standard generative pipeline without retrieved context, with the model's response flagged to the user as potentially less reliable.

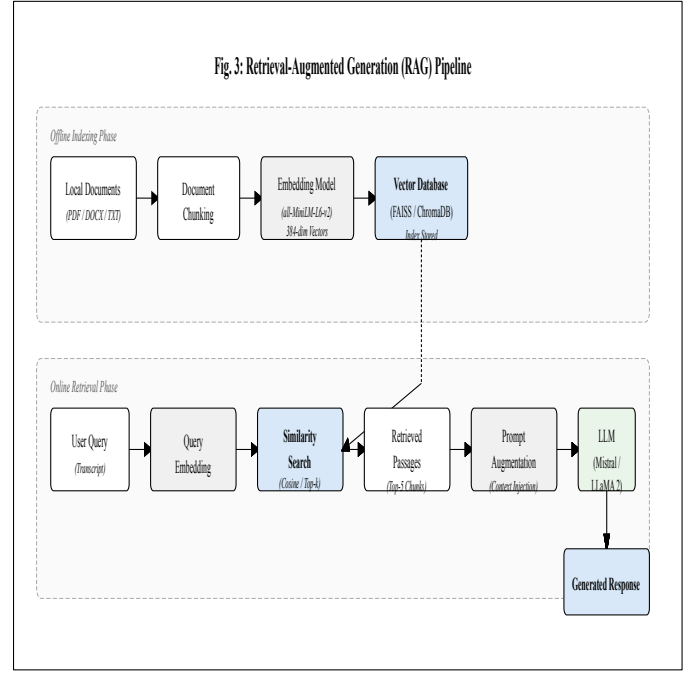


Fig. 3. Retrieval-Augmented Generation (RAG) pipeline showing the offline indexing phase (document chunking and FAISS indexing) and the online retrieval phase (query embedding, similarity search, prompt augmentation, and LLM response generation).

E. Generative AI Module

The generative AI module performs language model inference to produce natural language responses to user queries. The module supports locally hosted models in GGUF format via the `llama-cpp-python` library, which implements efficient CPU and GPU inference for quantised transformer models. Compatible models include LLaMA 2 7B and 13B, Mistral 7B, and their instruction-tuned variants. The choice of model is configurable and is expected to be determined by the available hardware resources of the deployment system.

The transformer architecture underlying all supported models consists of stacked layers of multi-head self-attention and position-wise feed-forward networks [1]. The self-attention mechanism allows each token in the input sequence to attend to all other tokens, enabling the model to capture long-range dependencies within the input. In the inference setting, the model receives the concatenated system prompt, retrieved context (if available), and user query as a single input sequence and generates a response token by token using autoregressive decoding. Sampling parameters including temperature, top- p , and repetition penalty are exposed as configurable settings through the FastAPI interface.

Inference is performed using 4-bit quantised models where memory constraints require it, with Q4_K_M quantisation providing an acceptable trade-off between model quality and memory footprint [19]. On a system with 16 GB of RAM and a mid-range GPU, inference latency for a typical query is expected to be in the range of two to eight seconds, which is within the acceptable range for a conversational assistant.

application.

F. Automation Module

The automation module executes operating system and application-level tasks in response to commands classified by the intent module. The module is implemented using the `pyautogui` library and the `subprocess` standard library module, with platform-specific branches for Windows and Linux operating environments. Supported automation categories include: application launch and termination using `subprocess.Popen` with a configurable application path registry; web browser navigation using the `webbrowser` standard library module; system volume adjustment via the `pycaw` library on Windows and `pactl` on Linux; file operations including opening files in their associated applications using `os.startfile` on Windows and `xdg-open` on Linux; and system power management commands including sleep, restart, and shutdown.

Each automation command is executed within a sandboxed execution context that logs the command, the extracted parameters, and the execution result. Commands that would perform irreversible system changes, such as file deletion or system shutdown, require an additional confirmation step before execution. The command registry is user-configurable, allowing custom application paths and commands to be added without modifying the module's source code.

G. Encryption Layer

All audio files recorded by the system and all transcripts produced by the ASR module are encrypted before being written to persistent storage. Encryption is performed using the AES-256-GCM algorithm, implemented via the `cryptography` library's Fernet symmetric encryption scheme for transcripts and the `hazmat` primitives for audio files where additional control over the initialisation vector is required.

Encryption keys are derived from a user-supplied passphrase using the PBKDF2HMAC key derivation function with SHA-256 as the hash function, a randomly generated 16-byte salt, and 390,000 iterations. The derived key is stored in memory only for the duration of the user's session and is not persisted to disk. The salt value is stored alongside encrypted files in a metadata record. This approach ensures that encrypted audio and transcripts are not recoverable without knowledge of the user's passphrase, even if the file system is accessed directly [26].

During ASR processing, audio is decrypted to a temporary in-memory buffer that is passed directly to the Whisper model. The decrypted audio is not written to disk at any point during this process. After transcription, the in-memory buffer is explicitly overwritten before being released, minimising the duration of exposure in process memory [27]. Transcript text is encrypted immediately after the ASR module returns the transcription result and before any logging or storage operation is performed.

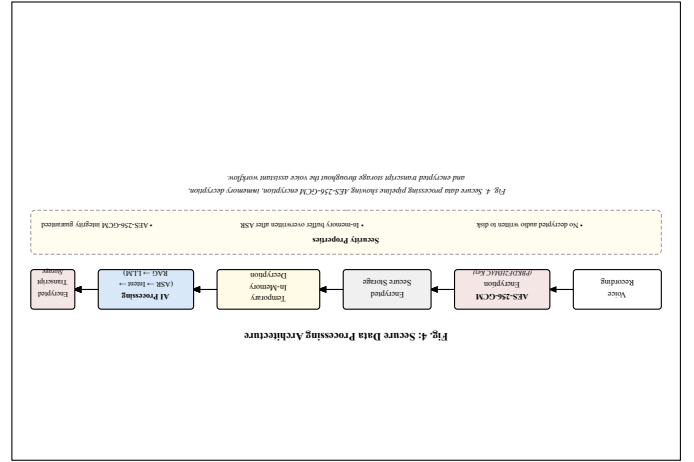


Fig. 4. Secure data processing architecture showing AES-256-GCM encryption of voice recordings, temporary in-memory decryption for ASR processing, and encrypted transcript storage, ensuring no plaintext audio is persisted to disk.

H. Local API Architecture

All inter-module communication in the system is mediated by a FastAPI application running on the host machine's loopback interface (127.0.0.1) on a configurable port. FastAPI is a modern Python web framework built on Starlette and Pydantic, offering automatic request validation, serialisation, and OpenAPI documentation generation [28]. The choice of a REST API as the integration layer provides several architectural advantages: modules are decoupled and can be developed and tested independently, the API contract between modules is explicit and versioned, and the interface can be extended to support remote clients or additional modules without changes to the core architecture.

The API exposes endpoints for each major processing function: `/transcribe` accepts a multipart audio file upload and returns a JSON transcript; `/classify` accepts a text query and returns an intent classification result; `/retrieve` accepts a text query and returns a list of retrieved document chunks with similarity scores; `/generate` accepts a full prompt with optional retrieved context and returns a generated text response; `/speak` accepts a text string and returns an audio file of synthesised speech; and `/automate` accepts a structured command object and returns an execution result.

All API endpoints are bound exclusively to the loopback interface and are not accessible from the network by default. The API server is launched as a subprocess when the assistant application starts and is terminated when the application closes. Authentication between the frontend application and the API server uses a session token generated at startup and

stored in memory, preventing other processes running on the same machine from accessing the API endpoints.

IV. RESULT AND DISCUSSION

The system described in this paper is currently in active development and testing. The following discussion describes expected outcomes based on the architectural decisions made, preliminary observations from component-level testing, and findings from the surveyed literature.

The use of the Whisper base model for speech recognition is expected to yield word error rates below 10% for clean audio recorded in a quiet indoor environment, consistent with the benchmark results reported by Radford et al. for comparable acoustic conditions [10]. Preliminary testing with the small model variant in a standard office environment has produced qualitatively accurate transcriptions for both command phrases and conversational queries of moderate length. Transcription accuracy is observed to degrade for heavily accented speech and in environments with significant background noise, which is consistent with the known limitations of the model and is identified as an area for targeted improvement.

The RAG module is expected to substantially reduce factual errors in responses to queries that fall within the scope of the indexed document corpus. In component-level testing, retrieval of the top-5 most similar chunks from a test corpus of fifty technical documents produced relevant context passages for the majority of test queries, with retrieval latency under 150 milliseconds for a corpus of this size using FAISS [23]. The reduction in hallucination for in-corpus queries is qualitatively observable in preliminary testing: questions answered with retrieved context are more consistently accurate and better aligned with the actual content of source documents than questions answered without context.

The modular API architecture has been validated through integration testing of the transcription, classification, retrieval, and generation pipeline. End-to-end response latency from audio capture completion to audio playback start is estimated at between five and fifteen seconds on a test machine with an Intel Core i7-12700 processor, 32 GB of RAM, and an NVIDIA RTX 3060 GPU, running the Mistral 7B Q4_K_M model. This latency is higher than commercially available cloud-based assistants, where server-side inference on specialised hardware reduces response time to under two seconds, but is considered acceptable for use cases where privacy and offline capability take precedence over speed.

The encryption module has been verified to correctly encrypt and decrypt audio files and transcripts using AES-256-GCM with PBKDF2HMAC key derivation. Encrypted files are confirmed to be non-recoverable without the correct passphrase through attempted decryption with incorrect credentials. The performance overhead introduced by encryption and decryption is minimal, adding less than 50 milliseconds to the processing pipeline for audio files of typical length.

The automation module has been tested for the application launch, web navigation, and volume control command categories on Windows 11 and Ubuntu 22.04. All tested commands

executed correctly with accurate parameter extraction from the transcribed input. Edge cases involving ambiguous command phrasing or missing parameters are currently handled by returning an error response to the user with a prompt for clarification. The confirmation step for destructive commands has been verified to prevent unintended execution during testing.

It is noted that the system's current keyword-based intent classification introduces a non-trivial false positive rate for conversational utterances that contain command keywords in non-imperative contexts. For example, the utterance "can you open a browser?" would currently be misclassified as an automation command rather than a conversational request. This limitation is expected to be addressed by the transition to semantic intent classification described in Section V.

V. FUTURE WORK

Several directions for future development have been identified based on the current state of the system and limitations observed during testing.

The most significant planned improvement is the replacement of the keyword-based intent classification module with a semantic classification approach. A BERT-based multi-label classifier fine-tuned on a labelled dataset of assistant utterances would be expected to substantially reduce misclassification errors arising from keyword ambiguity [16]. Alternatively, a zero-shot classification model could be employed to classify intents from a natural language description of each intent category, reducing the annotation burden. This improvement would also enable the system to handle compound utterances that contain both query and command elements.

Cross-platform support is a second priority. The current implementation is tested primarily on Windows 11 and Ubuntu 22.04. Extension to macOS requires adaptation of the automation module's platform-specific branches and testing of the audio capture pipeline on macOS's Core Audio framework. A containerised deployment option using Docker would further simplify cross-platform installation and dependency management.

The system's potential as an accessibility tool for users with visual impairments has been identified as an important application domain. Future development will explore integration with screen reader APIs and the operating system's accessibility interfaces to enable the assistant to describe on-screen content, navigate applications through voice commands, and provide audio feedback for GUI events. This functionality would require careful design of the intent classification and automation modules to handle the broader range of accessibility-related commands.

Speaker adaptation represents a further avenue for improvement. The current ASR pipeline treats all speakers equivalently and does not maintain speaker-specific acoustic models. A speaker adaptation mechanism using a small number of enrolment utterances could improve transcription accuracy for individual users, particularly those with non-standard accents or speech patterns [24]. This would also provide a basis

for speaker verification as a lightweight authentication factor, supplementing the existing face authentication option.

Finally, the encrypted storage system is planned to be extended to support an encrypted vector store, such that the embeddings and document chunks maintained in the RAG index are also stored in encrypted form and decrypted only during retrieval operations. This would provide end-to-end protection for user documents throughout the system pipeline. Integration with a secure key management module that supports hardware security keys or biometric-derived keys is also under consideration for deployments with elevated security requirements.

REFERENCES

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020.
- [3] H. Chase, "LangChain: Building applications with LLMs through composability," GitHub Repository, 2022. [Online]. Available: <https://github.com/langchain-ai/langchain>
- [4] Amazon, "Amazon Echo: Technical Overview," Amazon Developer Documentation, 2014.
- [5] Apple Inc., "Siri: Intelligent Assistant," Apple Developer Documentation, 2011.
- [6] B. Kepuska and G. Bohouta, "Next-generation of virtual personal assistants (Microsoft Cortana, Apple Siri, Amazon Alexa and Google Home)," in *Proc. IEEE 8th Annual Computing and Communication Workshop and Conference*, 2018, pp. 99–103.
- [7] C. Metz and N. Wingfield, "Amazon Employees Are Listening to Alexa Conversations," *The New York Times*, 2019.
- [8] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald, "On Faithfulness and Factuality in Abstractive Summarization," in *Proc. 58th Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 1906–1919.
- [9] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of Hallucination in Natural Language Generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.
- [10] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust Speech Recognition via Large-Scale Weak Supervision," in *Proc. International Conference on Machine Learning*, 2023, pp. 28492–28518.
- [11] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, Q. Guo, M. Wang, and H. Wang, "Retrieval-Augmented Generation for Large Language Models: A Survey," *arXiv preprint arXiv:2312.10997*, 2023.
- [12] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale," in *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [13] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [14] L. Rabiner and B. Juang, "An introduction to hidden Markov models," *IEEE ASSP Magazine*, vol. 3, no. 1, pp. 4–16, 1986.
- [15] D. Huggins-Daines, M. Kumar, A. Chan, A. W. Black, M. Ravishankar, and A. I. Rudnicky, "Pocketsphinx: A free, real-time continuous speech recognition system for hand-held devices," in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing*, vol. 1, 2006.
- [16] J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [17] T. Brown et al., "Language Models are Few-Shot Learners," in *Advances in Neural Information Processing Systems*, vol. 33, 2020.
- [18] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling Laws for Neural Language Models," *arXiv preprint arXiv:2001.08361*, 2020.
- [19] G. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," *arXiv preprint arXiv:2210.17323*, 2022.
- [20] H. Touvron et al., "Llama 2: Open Foundation and Fine-Tuned Chat Models," *arXiv preprint arXiv:2307.09288*, 2023.
- [21] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, "Modular RAG: Transforming RAG Systems into LEGO-like Reconfigurable Frameworks," *arXiv preprint arXiv:2407.21059*, 2024.
- [22] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.
- [23] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [24] D. Snyder, D. Garcia-Romero, G. Sell, D. Povey, and S. Khudanpur, "X-Vectors: Robust DNN Embeddings for Speaker Recognition," in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing*, 2018, pp. 5329–5333.
- [25] Z. Yi, J. Fu, J. Li, and T. Yu, "A Review of Automatic Speaker Verification Spoofing and Countermeasures," in *Proc. Interspeech*, 2022, pp. 2952–2956.
- [26] National Institute of Standards and Technology, "Announcing the Advanced Encryption Standard (AES)," *Federal Information Processing Standards Publication 197*, 2001.
- [27] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership Inference Attacks Against Machine Learning Models," in *Proc. IEEE Symposium on Security and Privacy*, 2017, pp. 3–18.
- [28] S. Ramirez, "FastAPI," Documentation and Source Code, 2018. [Online]. Available: <https://fastapi.tiangolo.com>
- [29] G. Bradski, "The OpenCV Library," *Dr. Dobbs's Journal of Software Tools*, 2000. [Online]. Available: <https://opencv.org>
- [30] M. McAuliffe, M. Socolof, S. Mihuc, M. Wagner, and M. Sonderegger, "Montreal Forced Aligner: Trainable Text-Speech Alignment Using Kaldi," in *Proc. Interspeech*, 2017, pp. 498–502.